UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO
SCHOOL OF ENGINEERING UNIDAD DE POSGRADO
Facultad de Ingeniería en Ciencias de la Computación y Telecomunicaciones.

Diplomado: Cloud Infrastructure & DevSecOps
Modulo: Arquitectura de Nube y Redes Empresariales
Actividad: Laboratorio 2 – Tablas de ruteo y conectividad entre subredes.
Docente: Msc. Diego Alejandro Rivera Montaño
Estudiante: Elmer Fuentes Lozano
Fecha: 11/10/2026

SANTA CRUZ – BOLIVIA

Propósito y alcance

Este informe desarrolla el diseño de la red de una empresa de comercio electrónico y documenta cómo se configuraría en Amazon Web Services. El resultado propuesto es una VPC 10.0.0.0/16 distribuida en dos zonas de disponibilidad, con seis subredes y tablas de ruteo separadas para la capa web y las capas internas. Solo las dos subredes web tendrán una ruta directa a Internet; aplicación y datos conservarán únicamente conectividad interna en esta etapa.

El trabajo sigue exclusivamente la Ruta B. Los cálculos, las decisiones de arquitectura y los diagramas son ejercicios propios; los procedimientos y comandos se contrastaron con documentación oficial de AWS. Aunque el estudiante dispone de una cuenta AWS, no se desplegaron recursos ni se ejecutaron pruebas de red para este laboratorio. Las verificaciones se describen como resultados esperados, nunca como resultados observados.

Evidencia documental incorporada. Se incluyen seis capturas reales de páginas de documentación oficial de AWS, proporcionadas para este informe e identificadas individualmente como E1–E6, con su URL de origen. Son referencias para la Ruta B, no capturas de recursos creados en la consola. La sección 8 reúne las imágenes y la comprobación de la rúbrica.

Organización del informe

1. Fundamentos conceptuales y diagnóstico
2. Cálculo VLSM manual y arquitectura base
3. Manual de creación de VPC y subredes
4. Estrategia de ruteo y arquitectura completa
5. Manual de Route Tables e Internet Gateway
6. Verificaciones y resolución de problemas
7. Reflexión final y continuidad del proyecto
8. Evidencia documental y control de rúbrica
9. Referencias oficiales

I. Fundamentos conceptuales

1. ¿Qué es una tabla de ruteo dentro de una VPC y qué rol cumple la "ruta local" que el proveedor agrega automáticamente? Expliquen por qué esa ruta nunca debe eliminarse.

Una tabla de ruteo es el conjunto de decisiones que utiliza el router virtual de la VPC para dirigir paquetes según su IP de destino. Cada entrada combina un prefijo de red con un destino lógico o target. No decide qué aplicación puede entrar: esa autorización corresponde a controles como Security Groups y Network ACLs. En mi diseño, una misma solicitud puede tener una ruta válida y aun así ser rechazada por una regla de seguridad. [1, 26, 27]

AWS agrega automáticamente la ruta 10.0.0.0/16 → local en las tablas de esta VPC. Así, un servidor web de 10.0.0.0/23 puede dirigirse a una IP de aplicación de 10.0.4.0/24 sin pasar por Internet. No tengo que crear seis rutas para comunicar las seis subredes, porque todas pertenecen al mismo /16. El permiso del tráfico y su respuesta se revisan por separado. [2]

La indicación de la guía de no eliminar la ruta local protege esa conectividad básica. Técnicamente, AWS no permite borrarla como una ruta estática ordinaria, aunque sí permite reemplazar su target en diseños avanzados con dispositivos de inspección y luego restaurarlo. Aquí debe permanecer como local: cambiarlo sin diseñar el camino de ida y vuelta podría interrumpir la comunicación entre las capas. [3]

2. Expliquen qué significa la ruta 0.0.0.0/0 y por qué agregarla en la tabla de ruteo de una subred, apuntando al Internet Gateway, es lo que efectivamente "hace pública" a esa subred — no el simple hecho de que tenga asignada una dirección IP pública.

El prefijo /0 no fija ningún bit de red: coincide con cualquier destino IPv4. Es una ruta de último recurso, usada cuando no existe otra coincidencia más específica. Si el destino es 10.0.4.10, gana 10.0.0.0/16; si es 8.8.8.8, en este diseño gana 0.0.0.0/0. Por tanto, la ruta local y la ruta por defecto son decisiones alternativas según el destino, no dos saltos consecutivos obligatorios. [4]

La asociación con una tabla que envía tráfico al Internet Gateway hace pública a la subred. Una IP pública identifica un recurso hacia el exterior, pero no crea el camino de salida. Para que una EC2 se comunique directamente mediante IPv4 se necesitan además una IP pública o Elastic IP, un IGW adjunto y reglas que permitan el tráfico. Una subred puede ser pública aunque alguna instancia dentro de ella no tenga IP pública. [5, 6]

3. En AWS, una subred no es pública o privada "por naturaleza": lo es según la tabla de ruteo con la que esté asociada. Justifiquen esta afirmación explicando qué pasaría si, por error, se asociara la tabla de ruteo pública a una subred que debía quedar privada

Los nombres sn-app y sn-datos expresan mi intención, pero no imponen aislamiento. Si asocio por error sn-datos-zona-a con rt-publica-ecommerce, esa subred obtiene una ruta directa al IGW y pasa a ser pública por su ruteo. Eso incumple el diseño incluso si sus recursos todavía carecen de IP pública. No significa que todos se vuelvan accesibles de inmediato: siguen influyendo las direcciones y los filtros. El riesgo es dejar preparado un camino externo que podría activarse con un cambio posterior. [5, 7]

Para corregirlo comprobaría la pestaña Route table de la subred, volvería a asociarla con rt-privada-ecommerce y revisaría las otras cinco asociaciones. También comprobaría las IP públicas y reglas de seguridad de cualquier recurso ya desplegado; arreglar la asociación no sustituye esa revisión.

4. Comparen usar una sola tabla de ruteo para todas las subredes de la VPC contra usar tablas separadas por rol (pública/privada). Justifiquen cuál estrategia usarían en el escenario del proyecto (comercio electrónico con capas web, de aplicación y de datos) y por qué.

Estrategia	Ventaja	Consecuencia en el proyecto
Una tabla para las seis subredes	Menos objetos que administrar	Si incluye IGW, el ruteo público alcanza todas las capas; si solo tiene local, la capa web carece de salida directa.
Dos tablas personalizadas por rol	Expresa las dos políticas de conectividad	Permite salida directa solo a web y conserva aplicación y datos sin rutas externas.
Una tabla para cada subred	Permite políticas independientes	En Lab 2 repetiría rutas sin una necesidad funcional; puede ser útil al diferenciar salidas por zona.

Elegiría dos tablas personalizadas porque el criterio relevante hoy es la exposición de cada capa, no la cantidad de subredes. Ambas zonas tienen la misma política web y la misma política interna. Una tabla de subred puede aplicarse a varias subredes de la VPC, aunque estén en distintas AZ. Además dejaría la tabla principal automática solo con local y asociaría explícitamente las seis subredes; así una nueva subred no hereda accidentalmente una salida pública. [2, 8]

5. Expliquen para qué sirve un Internet Gateway y por qué una ruta 0.0.0.0/0 sin un Internet Gateway creado y adjunto a la VPC no tiene ningún efecto real sobre la conectividad.

El Internet Gateway es el componente de la VPC que permite intercambiar tráfico con Internet y que puede usarse como target de una ruta. En IPv4 participa en la traducción entre la dirección privada del recurso y su dirección pública. No es una EC2 que deba administrar ni es el NAT Gateway que se incorporaría después para salida desde subredes privadas. [5]

Escribir 0.0.0.0/0 no crea ese componente. Primero debe existir el IGW y estar adjunto a la VPC correcta. Un identificador inexistente o un gateway que no corresponde a la VPC no proporciona un destino de reenvío válido; la operación puede ser rechazada o la ruta puede quedar no utilizable según la situación. Por eso verificaría tanto el estado de la ruta como el vínculo del gateway, y no solo la presencia del texto /0. [6, 18]

6. ¿Qué es VPC Peering y en qué se diferencia de agregar una ruta hacia Internet? Describan un escenario del proyecto (por ejemplo, conectar la VPC de este ambiente con la de otro equipo o con un ambiente de producción distinto) donde tendría sentido usarlo, y qué rutas adicionales habría que agregar en las tablas de ruteo de ambos lados.

VPC Peering conecta dos VPC para intercambiar tráfico mediante sus direcciones privadas. Lo usaría, por ejemplo, si los servidores de aplicación del comercio electrónico debieran consultar un servicio interno de otro equipo en una VPC 10.1.0.0/16. Es un escenario hipotético adicional; no forma parte de las rutas activas del Lab 2. [9]

Lado	Tabla participante	Ruta adicional hipotética
Comercio electrónico 10.0.0.0/16	rt-privada-ecommerce	10.1.0.0/16 → conexión pcx del peering
Servicios de otro equipo 10.1.0.0/16	Tabla de la subred del servicio	10.0.0.0/16 → la misma conexión pcx

La ruta de retorno es necesaria: una ruta de ida no completa la comunicación. La conexión tendría que estar aceptada y activa, los bloques de ambas VPC no podrían solaparse y los filtros tendrían que permitir el servicio. El peering no crea salida general a Internet ni ofrece enrutamiento transitivo hacia cualquier tercera red. Para limitar el acceso real al servicio usaría reglas de seguridad específicas; el /16 de la tabla muestra el alcance del ejemplo, no una autorización de todos los puertos. [9, 10]

7. Si ejecutan traceroute desde una instancia de la subred pública hacia Internet y el primer salto no corresponde al gateway de su propia subred, ¿qué indicaría eso sobre la tabla de ruteo asociada? Expliquen cómo diagnosticarían el problema paso a paso.

En un diagnóstico básico, un primer salto inesperado podría sugerir una ruta equivocada, otra interfaz de salida o una subred asociada a la tabla incorrecta. Sin embargo, no concluiría que la Route Table está mal únicamente por esa observación. traceroute depende de respuestas a sondas y de mensajes ICMP de tiempo excedido; el primer equipo visible no tiene por qué representar todos los componentes lógicos que atravesó el paquete. AWS documenta reglas ICMP específicas que pueden afectar estas respuestas. [11]

Distingo dos verificaciones: la ruta por defecto del sistema operativo apunta normalmente a la dirección base de la subred más uno, reservada para el router de la VPC; la Route Table de AWS decide después el target según el destino. En sn-web-zona-a, esa dirección reservada es 10.0.0.1. La ausencia de ese valor en traceroute, o la aparición de asteriscos, no demuestra por sí sola una mala asociación. Tampoco demuestra que la ruta local /16 se use para alcanzar 8.8.8.8, que está fuera del /16. [4, 11, 12]

1. Identificar el origen. Comprobaría la IP privada, la interfaz y el ID de subred de la instancia. El nombre visible de una EC2 no asegura que esté en la subred web prevista.

2. Revisar el sistema operativo. Consultaría ip -4 address, ip -4 route e ip route get 8.8.8.8. Esperaría una interfaz de salida coherente y un siguiente salto dentro de la subred. Una VPN local o una ruta más específica podrían cambiar la decisión del host.

3. Revisar la asociación efectiva en AWS. Abriría VPC → Subnets → subred de origen → Route table. Si no hay asociación explícita, comprobaría la tabla principal. Para web debe ser rt-publica-ecommerce.

4. Revisar prefijos y target. Confirmaría 10.0.0.0/16 → local y 0.0.0.0/0 → IGW en estado active, además de cualquier ruta más específica que pudiera ganar sobre /0.

5. Revisar los requisitos restantes. Comprobaría el IGW adjunto, la IP pública de la instancia, Security Group, NACL y firewall del sistema. Un fallo de SSH también puede involucrar ruteo; no lo atribuiría automáticamente al firewall.

6. Comparar varias pruebas. Usaría ping y traceroute con límites, y una conexión TCP como prueba complementaria. Si las sondas no responden pero TCP sí, investigaría filtros ICMP antes de cambiar rutas.

7. Confirmar la causa. Si persistiera la duda, un análisis de configuración con Reachability Analyzer permitiría localizar un bloqueo lógico. Es una posibilidad futura, no una prueba realizada; este servicio analiza configuración y no envía paquetes. [13]

```
ip -4 address
ip -4 route
ip route get 8.8.8.8
ping -c 4 -W 2 8.8.8.8
traceroute -n -m 12 -w 2 8.8.8.8
```

Los comandos anteriores son un plan de diagnóstico para Linux con las utilidades instaladas. No se adjuntan salidas de terminal porque no se ejecutaron en una instancia del proyecto. La asociación efectiva se verifica según el procedimiento oficial de AWS. [8]

II. Cálculo VLSM manual

2.1 Datos de entrada y método

El bloque 10.0.0.0/16 tiene 32 − 16 = 16 bits para numerar direcciones dentro de la VPC. Su tamaño es 2^{16} = 65 536 direcciones y abarca desde 10.0.0.0 hasta 10.0.255.255. La máscara del bloque padre es 255.255.0.0. Debo repartir seis subredes: 500, 500, 250, 250, 50 y 50 hosts. Para reducir el desperdicio asigno primero las subredes más grandes y después las menores.

En subnetting IPv4 tradicional, busco el menor número h de bits de host que cumpla $2^h - 2 \geq H$, donde H es el requisito. El prefijo será p = 32 − h. “Bloque más pequeño” significa menor cantidad de direcciones suficiente; por eso se elige el prefijo más largo que todavía cumple la capacidad. Para AWS comprobaré además $2^h - 5 \geq H$, porque se reservan cinco direcciones por subred. En este ejercicio ambas condiciones llevan a los mismos prefijos. [12]

2.2 Dimensionamiento por capa

1. Web con 500 hosts por zona. Con h = 8: $2^8 - 2 = 254$, insuficiente. Con h = 9: $2^9 - 2 = 510$, suficiente. Entonces p = 32 − 9 = 23, tamaño 512 direcciones, máscara 255.255.254.0. En AWS quedan 512 − 5 = 507 direcciones asignables, con margen de 7 sobre el requisito. Se necesitan dos bloques /23.

2. Aplicación con 250 hosts por zona. Con h = 7: $2^7 - 2 = 126$, insuficiente. Con h = 8: $2^8 - 2 = 254$, suficiente. Entonces p = 24, tamaño 256 direcciones y máscara 255.255.255.0. En AWS quedan 251 direcciones asignables, con margen de solo 1. Se necesitan dos bloques /24.

3. Datos con 50 hosts por zona. Con h = 5: $2^5 - 2 = 30$, insuficiente. Con h = 6: $2^6 - 2 = 62$, suficiente. Entonces p = 26, tamaño 64 direcciones y máscara 255.255.255.192. En AWS quedan 59 direcciones asignables, con margen de 9. Se necesitan dos bloques /26.

Capa	Hosts por AZ	Prefijo	Total	Útiles IPv4	Asignables AWS
Web	500	/23	512	510	507
Aplicación	250	/24	256	254	251
Datos	50	/26	64	62	59

La capa de aplicación cumple matemáticamente, pero su margen real es reducido. Los servicios administrados y las interfaces adicionales también consumen direcciones. Para conservar exactamente la arquitectura exigida mantengo /24; para un despliegue de producción evaluaría crecimiento y consumo por servicio antes de aprobarlo. Esta observación no cambia las seis redes del laboratorio.

Asignación paso a paso de los seis bloques

Para cada subred fijo la primera dirección alineada disponible, calculo el último valor como red + tamaño − 1 y asigno el siguiente bloque después de ese límite. Las operaciones siguientes son el desarrollo manual; no dependen de una calculadora VLSM.

1. Web de la Zona A. La máscara /23 tiene 254 en el tercer octeto: el salto es 256 − 254 = 2 en ese octeto. Inicio en 10.0.0.0, múltiplo válido del salto. Sus 512 direcciones ocupan los terceros octetos 0 y 1. El último valor es 10.0.1.255. El rango tradicional de hosts va de 10.0.0.1 a 10.0.1.254. La siguiente red disponible es 10.0.2.0.

2. Web de la Zona B. Asigno el siguiente /23 alineado: 10.0.2.0/23. El tercer octeto 2 es múltiplo de 2. El bloque ocupa los terceros octetos 2 y 3 y termina en 10.0.3.255. Sus hosts tradicionales van de 10.0.2.1 a 10.0.3.254. La primera dirección libre siguiente es 10.0.4.0.

3. Aplicación de la Zona A. Un /24 abarca 256 direcciones; avanza una unidad en el tercer octeto y empieza con el cuarto octeto en cero. Tomo 10.0.4.0/24. Red + 255 = 10.0.4.255, su broadcast teórico. Hosts tradicionales: 10.0.4.1 a 10.0.4.254. El siguiente inicio es 10.0.5.0.

4. Aplicación de la Zona B. Asigno 10.0.5.0/24 con el mismo tamaño. El bloque termina en 10.0.5.255; los hosts tradicionales van de 10.0.5.1 a 10.0.5.254. No invade el /24 anterior, porque empieza después de 10.0.4.255. La siguiente dirección libre es 10.0.6.0.

5. Datos de la Zona A. En /26 la máscara tiene 192 en el cuarto octeto. El salto es 256 − 192 = 64; los inicios válidos dentro de 10.0.6.0/24 son .0, .64, .128 y .192. Tomo el primero, 10.0.6.0/26. Red + 63 = 10.0.6.63. Hosts tradicionales: 10.0.6.1 a 10.0.6.62. El siguiente inicio es 10.0.6.64.

6. Datos de la Zona B. Tomo 10.0.6.64/26. Como 64 es múltiplo del salto, está correctamente alineado. Último valor: 64 + 63 = 127, es decir 10.0.6.127. Hosts tradicionales: 10.0.6.65 a 10.0.6.126. El espacio libre comienza en 10.0.6.128.

2.3 Comprobación de alineación y uso del bloque

Las máscaras en binario son:

/23 = 11111111.11111111.11111110.00000000

/24 = 11111111.11111111.11111111.00000000

/26 = 11111111.11111111.11111111.11000000

En cada dirección de red, todos los bits de host son cero; para obtener el broadcast teórico se ponen en uno.

Consumo total: 2 × 512 + 2 × 256 + 2 × 64 = 1 664 direcciones. Espacio sin asignar: 65 536 − 1 664 = 63 872 direcciones. Los bloques son contiguos de 10.0.0.0 a 10.0.6.127 y todos quedan dentro del /16. Como el inicio de cada bloque supera el final del anterior, no existen solapamientos. El intervalo libre 10.0.6.128–10.0.255.255 es una reserva de espacio, no una única subred CIDR ya creada.

Tablas finales de direccionamiento

La primera tabla completa los campos de la plantilla con el criterio IPv4 tradicional. La segunda precisa cuáles de esas direcciones puede asignar AWS a recursos. En AWS no se utiliza broadcast; se informa su valor matemático porque la guía lo solicita. [12]

Capa y requisito	CIDR	Red	Broadcast teórico	Hosts tradicionales	Rol previsto
Web A 500 hosts	10.0.0.0/23	10.0.0.0	10.0.1.255	10.0.0.1 a 10.0.1.254	Pública
Web B 500 hosts	10.0.2.0/23	10.0.2.0	10.0.3.255	10.0.2.1 a 10.0.3.254	Pública
Aplicación A 250 hosts	10.0.4.0/24	10.0.4.0	10.0.4.255	10.0.4.1 a 10.0.4.254	Privada
Aplicación B 250 hosts	10.0.5.0/24	10.0.5.0	10.0.5.255	10.0.5.1 a 10.0.5.254	Privada
Datos A 50 hosts	10.0.6.0/26	10.0.6.0	10.0.6.63	10.0.6.1 a 10.0.6.62	Privada
Datos B 50 hosts	10.0.6.64/26	10.0.6.64	10.0.6.127	10.0.6.65 a 10.0.6.126	Privada

Subred	Rango realmente asignable en AWS	Capacidad AWS	Margen
Web A	10.0.0.4 a 10.0.1.254	507	7
Web B	10.0.2.4 a 10.0.3.254	507	7
Aplicación A	10.0.4.4 a 10.0.4.254	251	1
Aplicación B	10.0.5.4 a 10.0.5.254	251	1
Datos A	10.0.6.4 a 10.0.6.62	59	9
Datos B	10.0.6.68 a 10.0.6.126	59	9

Las cinco direcciones reservadas

Para una subred de dirección base N, AWS reserva N, N+1, N+2, N+3 y la última dirección. Representan la red, el router de la VPC, la reserva relacionada con DNS, una reserva para uso

futuro y el broadcast no soportado. Por ejemplo, en Datos B son 10.0.6.64, 10.0.6.65, 10.0.6.66, 10.0.6.67 y 10.0.6.127. No se restan dos y luego cinco: cinco es el total de reservas de AWS. [12]

La dirección DNS del rango primario de esta VPC es 10.0.0.2. Reservar N+2 en cada subred no significa que cada una tenga un servidor DNS independiente allí. Esta diferencia evita confundir una dirección reservada con la dirección que debe usarse para un servicio. [12]

Arquitectura base y exposición de las capas

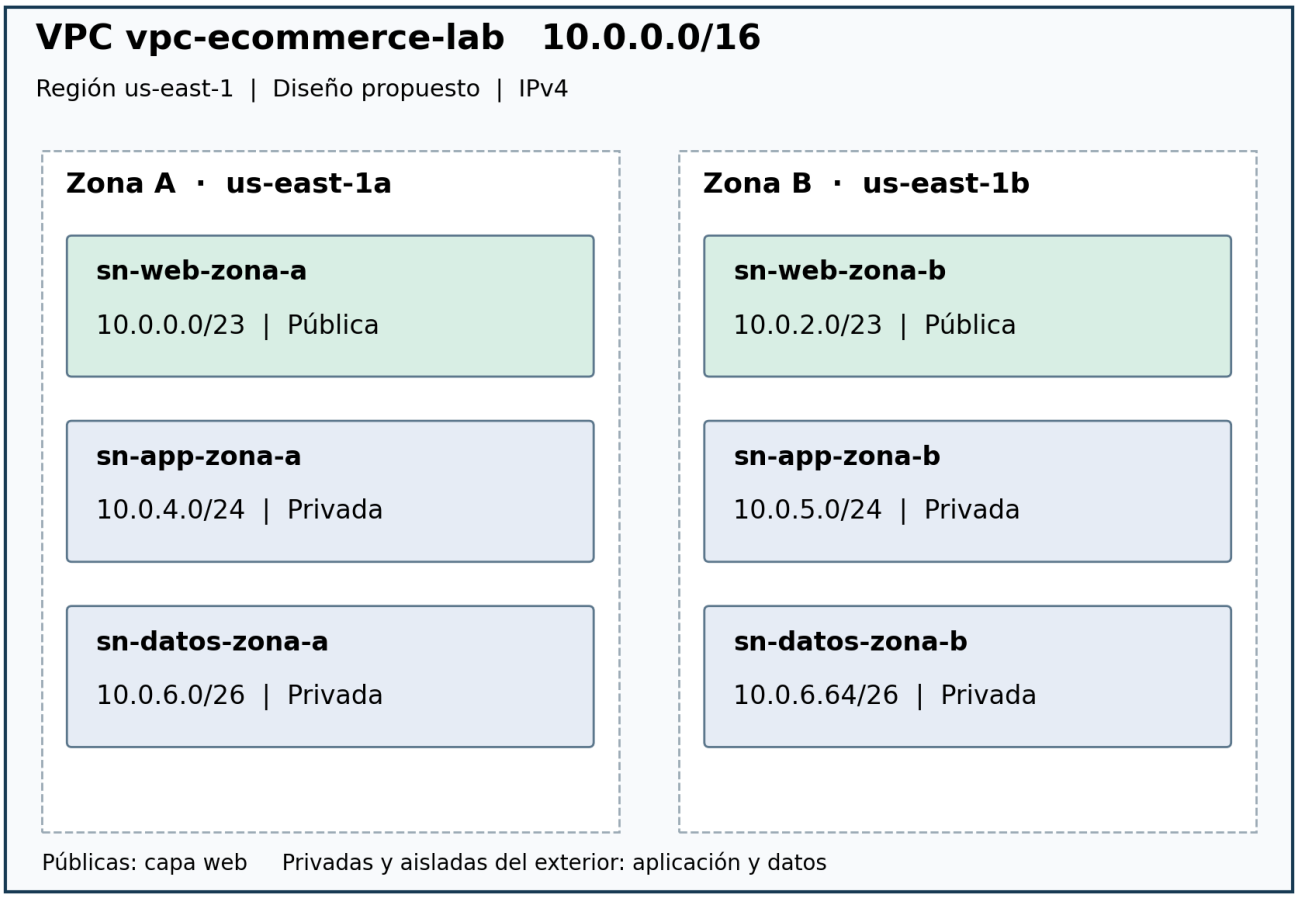


Figura 1. Arquitectura propia derivada del cálculo VLSM. Los rótulos pública y privada expresan el estado previsto tras configurar las asociaciones.

Justificación de las subredes públicas y privadas

Las dos subredes web deben ser públicas porque alojarían el punto de atención de las solicitudes de los clientes. En cambio, la aplicación ejecutaría la lógica del negocio y la capa de datos conservaría información que no necesita una entrada directa desde Internet. Mantener estas cuatro subredes privadas reduce las vías de exposición y facilita aplicar controles diferentes a cada función. La división no elimina la necesidad de permitir solo los flujos internos requeridos: disponer de conectividad entre capas no autoriza cualquier puerto. Se replica una subred de cada capa en cada zona para poder distribuir los componentes; tener subredes en dos AZ prepara la redundancia, pero por sí solo no demuestra alta disponibilidad de una aplicación que todavía no existe.

III. Manual de creación de VPC y subredes

3.1 Qué se construye y qué se necesita

El CIDR de la VPC delimita el espacio privado disponible para sus recursos: no asigna automáticamente todas sus direcciones a máquinas ni anuncia este bloque privado a Internet. Cada subred usa una porción contenida en ese espacio y pertenece a una sola AZ. La VPC abarca la región; una subred no se extiende entre dos zonas. [7, 14]

Precondiciones para una ejecución futura: acceso autorizado a una cuenta, permisos EC2/VPC de creación y consulta, selección de us-east-1 y cuotas suficientes. Para la Ruta B no se requieren credenciales ni recursos. El procedimiento comienza en una VPC nueva, sin reutilizar una VPC predeterminada ni crear NAT. Se elige VPC only para controlar por separado los seis CIDR calculados. [15]

3.2 Procedimiento documentado en consola

1. Seleccionar región y servicio. En la consola de AWS escoger us-east-1 y abrir VPC. Resultado esperado: todas las pantallas siguientes corresponden a la misma región. Verificación: revisar el selector regional antes de crear objetos. Esto evita buscar después los recursos en otra región.

2. Crear la VPC. Ir a Your VPCs → Create VPC y elegir VPC only. Escribir Name tag = vpc-ecommerce-lab e IPv4 CIDR manual = 10.0.0.0/16. No añadir IPv6 para este ejercicio y mantener tenancy Default. Revisar los valores y seleccionar Create VPC. [15]

3. Comprobar la VPC. Esperar el estado Available y abrir sus detalles. Resultado esperado: un VPC ID generado por AWS y el CIDR 10.0.0.0/16. Registrar el ID real si se ejecutara, sin inventarlo. Verificar que el nombre no sea el único criterio de identificación, porque las etiquetas pueden repetirse.

4. Abrir el formulario de subredes. Ir a Subnets → Create subnet y seleccionar exactamente el VPC ID anterior. Para cada entrada especificar nombre, Availability Zone e IPv4 CIDR manual según la tabla de la siguiente página. No dejar la AZ a elección automática, porque la distribución por zonas forma parte del requisito. [16]

5. Crear las seis entradas. Introducir una fila por subred o repetir el formulario individualmente. Revisar el prefijo antes de confirmar cada una. Resultado esperado: seis Subnet ID distintos con estado Available. La creación documentada no incluye EC2, balanceadores ni bases de datos.

6. Revisar el inventario. Filtrar Subnets por el VPC ID. Comparar nombre, CIDR, AZ y VPC de las seis filas contra el diseño. Verificar que web A, aplicación A y datos A compartan us-east-1a y que las tres B usen us-east-1b. [16, 20]

7. Comprobar el estado inicial de ruteo. Antes de crear las tablas personalizadas, las subredes usan la tabla principal. En una VPC creada con VPC only se espera únicamente la ruta local. Ninguna es pública todavía por llamarse web. En este punto no se crea IGW ni se añade /0; esa configuración se documenta en la sección 5. [2]

Criterio de aceptación de este procedimiento: VPC correcta y seis CIDR/AZ coincidentes, sin solapamientos. Una captura de una consola con esos valores sería evidencia de ejecución de Ruta A; no se presenta como evidencia de esta investigación.

Inventario exacto y preparación de AWS CLI

Las letras de las AZ son nombres elegidos para este ejercicio. Antes de una ejecución se comprobaría que ambas estén disponibles en la cuenta y que correspondan a zonas distintas. No se afirma haber consultado una cuenta real. La asignación por nombre puede variar entre cuentas; para coordinar infraestructura entre cuentas se contrastarían los ZoneId. [19]

Nombre de subred	CIDR	AZ elegida	Variable del ID
sn-web-zona-a	10.0.0.0/23	us-east-1a	WEB_A
sn-web-zona-b	10.0.2.0/23	us-east-1b	WEB_B
sn-app-zona-a	10.0.4.0/24	us-east-1a	APP_A
sn-app-zona-b	10.0.5.0/24	us-east-1b	APP_B
sn-datos-zona-a	10.0.6.0/26	us-east-1a	DATOS_A
sn-datos-zona-b	10.0.6.64/26	us-east-1b	DATOS_B

3.3 Convenciones de los comandos

Los bloques están escritos para Bash, por ejemplo en Linux o WSL, con AWS CLI v2 instalada y autenticación ya configurada de forma segura. No son sintaxis de PowerShell. Se ejecutarían en orden y en la misma sesión: las variables guardan los identificadores que devolvería AWS. No se usan IDs ficticios como si fueran recursos reales. Las barras invertidas al final de una línea continúan el mismo comando y no deben llevar espacios después.

La región queda definida para todos los comandos mediante AWS_DEFAULT_REGION y AWS_REGION. El perfil se tomaría de la autenticación del entorno; si se necesitara otro, se configuraría antes de iniciar. set -e detiene la secuencia ante fallos no controlados. Es un guion de creación, no una herramienta idempotente: no debe repetirse completo para corregir una sola operación.

```
set -e
export AWS_DEFAULT_REGION=us-east-1
export AWS_REGION=us-east-1
export AWS_PAGER=""
aws --version
aws sts get-caller-identity
aws ec2 describe-availability-zones \
  --filters Name=state,Values=available \
  --query 'AvailabilityZones[].{AZ:ZoneName,ID:ZoneId}' \
  --output table
```

Resultado esperado: una versión 2.x de CLI, una identidad autorizada y una lista de AZ disponibles. Verificar us-east-1a y us-east-1b antes de continuar; si no están disponibles, el diseño y todos los comandos tendrían que ajustarse coherentemente. No se publicaría la salida con identificadores de cuenta innecesarios. describe-availability-zones consulta disponibilidad; no crea recursos. [19]

Creación de VPC y primeras subredes

3.4 Crear la VPC y esperar su disponibilidad

```
VPC_ID=$(aws ec2 create-vpc \
  --cidr-block 10.0.0.0/16 \
  --instance-tenancy default \
  --tag-specifications \
  'ResourceType=vpc,Tags=[{Key=Name,Value=vpc-ecommerce-lab}]' \
  --query 'Vpc.VpcId' --output text)
aws ec2 wait vpc-available --vpc-ids "$VPC_ID"
```

create-vpc solicita la VPC; --cidr-block define el espacio IPv4, --instance-tenancy default mantiene la tenencia compartida y --tag-specifications asigna el nombre. --query extrae únicamente Vpc.VpcId y --output text permite guardarlo en VPC_ID. El waiter termina cuando la VPC está disponible o devuelve un error si no se cumple la condición dentro de su espera. En caso de fallo, no se continúa con subredes. [17, 21]

Seis comandos independientes, uno por subred. --vpc-id usa la VPC recién creada, --cidr-block coincide exactamente con el cálculo y --availability-zone fija la zona. La etiqueta Name facilita la identificación, pero no determina la privacidad. [22]

3.5 Web A

```
WEB_A=$(aws ec2 create-subnet \
  --vpc-id "$VPC_ID" --cidr-block 10.0.0.0/23 \
  --availability-zone us-east-1a \
  --tag-specifications \
  'ResourceType=subnet,Tags=[{Key=Name,Value=sn-web-zona-a}]' \
  --query 'Subnet.SubnetId' --output text)
```

Web B

```
WEB_B=$(aws ec2 create-subnet \
  --vpc-id "$VPC_ID" --cidr-block 10.0.2.0/23 \
  --availability-zone us-east-1b \
  --tag-specifications \
  'ResourceType=subnet,Tags=[{Key=Name,Value=sn-web-zona-b}]' \
  --query 'Subnet.SubnetId' --output text)
```

Aplicación A

```
APP_A=$(aws ec2 create-subnet \
  --vpc-id "$VPC_ID" --cidr-block 10.0.4.0/24 \
  --availability-zone us-east-1a \
  --tag-specifications \
  'ResourceType=subnet,Tags=[{Key=Name,Value=sn-app-zona-a}]' \
  --query 'Subnet.SubnetId' --output text)
```

AWS CLI para las subredes restantes

Aplicación B

```
APP_B=$(aws ec2 create-subnet \
  --vpc-id "$VPC_ID" --cidr-block 10.0.5.0/24 \
  --availability-zone us-east-1b \
  --tag-specifications \
  'ResourceType=subnet,Tags=[{Key=Name,Value=sn-app-zona-b}]' \
  --query 'Subnet.SubnetId' --output text)
```

Datos A

```
DATOS_A=$(aws ec2 create-subnet \
  --vpc-id "$VPC_ID" --cidr-block 10.0.6.0/26 \
  --availability-zone us-east-1a \
  --tag-specifications \
  'ResourceType=subnet,Tags=[{Key=Name,Value=sn-datos-zona-a}]' \
  --query 'Subnet.SubnetId' --output text)
```

Datos B

```
DATOS_B=$(aws ec2 create-subnet \
  --vpc-id "$VPC_ID" --cidr-block 10.0.6.64/26 \
  --availability-zone us-east-1b \
  --tag-specifications \
  'ResourceType=subnet,Tags=[{Key=Name,Value=sn-datos-zona-b}]' \
  --query 'Subnet.SubnetId' --output text)
```

Resultado esperado de cada comando: un SubnetId real guardado en su variable. El estado puede pasar brevemente por pending antes de available. Las subredes estarán vacías; no se consumen aquí direcciones de interfaces de EC2. Si AWS rechaza una entrada, se revisan VPC, CIDR y AZ antes de reintentar únicamente esa operación.

3.6 Verificar la creación con consultas

```
aws ec2 describe-vpcs --vpc-ids "$VPC_ID" \
  --query 'Vpcs[].{ID:VpcId,CIDR:CidrBlock,Estado:State}' \
  --output table
aws ec2 describe-subnets \
  --filters "Name=vpc-id,Values=$VPC_ID" \
  --query 'Subnets[].{ID:SubnetId,CIDR:CidrBlock,AZ:AvailabilityZone,Estado:State}' \
  --output table
```

La consulta de subredes debe devolver seis registros. Antes de configurar rutas se compararía cada CIDR y AZ con el inventario; si aún figura pending se repetiría la consulta hasta que esté disponible. --filters limita la búsqueda a la VPC del proyecto y --query selecciona campos de la respuesta sin modificar recursos. [20]

Validación y reflexión de VPC y subredes

3.7 Lista técnica de aceptación

Comprobación	Resultado esperado	Razón
Bloque padre	10.0.0.0/16 en estado available	Es la base única de direccionamiento.
Cantidad y pertenencia	Seis subredes con el mismo VPC ID	Evita mezclar recursos de otra VPC.
Asignación por zona	Tres en us-east-1a y tres en us-east-1b	Respeta las dos ubicaciones previstas.
CIDR y alineación	Dos /23, dos /24 y dos /26 exactos	El prefijo cambia capacidad y límites.
Capacidad inicial vacía	507, 507, 251, 251, 59 y 59 IPv4 libres	Descuenta las cinco reservas por subred.
Ruteo inicial	Tabla principal con 10.0.0.0/16 → local	Todavía no se habilita salida a Internet.

3.8 Reflexión sobre validación y errores de creación

Después de crear la VPC y las subredes, mi primera validación sería comparar lo registrado en AWS con el cálculo, sin asumir que una creación exitosa significa que el diseño está bien. Revisaría el bloque 10.0.0.0/16, el identificador de la VPC y las seis combinaciones de nombre, CIDR y zona. También comprobaría que el inicio y el final de cada subred permanezcan dentro del bloque padre y que ninguna se superponga con otra. La cantidad de direcciones disponibles me ayudaría a detectar un prefijo copiado incorrectamente; por ejemplo, un /26 no puede reemplazar al /24 de aplicación aunque ambos empiecen por 10.0.

AWS documenta InvalidSubnet.Conflict cuando el CIDR entra en conflicto con otra subred, e InvalidSubnet.Range para un rango inválido. También distingue identificadores de VPC inexistentes o mal formados. Por ello revisaría primero los datos de entrada y la región, en vez de repetir la creación sin cambios. Finalmente verificaría que las AZ seleccionadas existan en la cuenta y que cada capa esté distribuida como se planeó. Esta revisión me parece necesaria porque un error de direccionamiento se arrastraría a las tablas de ruteo y a los siguientes laboratorios. [18]

Reflexión desarrollada por encima del mínimo de 100 palabras. Los valores de capacidad son expectativas calculadas para subredes vacías, no mediciones de una cuenta.

IV. Estrategia de ruteo

4.1 Tablas que necesita el diseño

Crearé documentalmente dos tablas personalizadas: rt-publica-ecommerce para las dos web y rt-privada-ecommerce para las cuatro internas. AWS también crea una tabla principal con la VPC; por tanto, en una ejecución se esperarían tres tablas en total, no solamente dos. La principal se conserva con local, sin asociaciones explícitas del proyecto. [2]

Tabla	Subredes asociadas	Destino o prefijo	Target
rt-publica-ecommerce	sn-web-zona-a sn-web-zona-b	10.0.0.0/16	local
rt-publica-ecommerce	sn-web-zona-a sn-web-zona-b	0.0.0.0/0	IGW identificado por IGW_ID Nombre igw-ecommerce
rt-privada-ecommerce	sn-app-zona-a sn-app-zona-b sn-datos-zona-a sn-datos-zona-b	10.0.0.0/16	local
Tabla principal automática	Ninguna asociación explícita prevista	10.0.0.0/16	local

La tabla reúne una fila por ruta para que destino y target no resulten ambiguos. Dos filas pertenecen a la misma tabla pública; no representan dos tablas diferentes. Los nombres de IGW y tablas son etiquetas del diseño. En una consulta real, AWS mostraría identificadores igw- y rtb- generados por el servicio.

4.2 Justificación del diseño y continuidad hacia NAT

Separar las tablas conserva el mismo criterio de exposición en ambas zonas. Con una sola tabla pública, un cambio para facilitar el acceso de los servidores web también modificaría el camino de salida de datos y aplicación. Con la separación puedo revisar y modificar la política de una función sin ampliar automáticamente la conectividad de todas las demás. La asociación explícita es tan importante como el contenido de la tabla: una ruta correcta en una tabla que ninguna subred web utiliza no resuelve la salida de esas subredes.

En las capas privadas no agrego 0.0.0.0/0 porque en Lab 2 no existe un componente autorizado para su salida a Internet. Esto deja esas subredes aisladas del exterior, manteniendo las rutas internas. No es segmentación por puertos: la ruta local también permite plantear tráfico entre web y datos, y los controles de seguridad del Lab 3 serán necesarios para limitarlo a los flujos del negocio. [7, 26, 27]

Cuando se estudie NAT en el Lab 3, el cambio conceptual será dar salida iniciada desde subredes privadas mediante un target NAT, conservando su ausencia de ruta directa al IGW. Si se adopta

salida por AZ, podrían requerirse tablas privadas por zona. Esa ampliación se decidirá allí; no cambia ahora la VPC ni sus CIDR y no se presenta como un recurso ya creado. [23]

Diagrama de ruteo completo

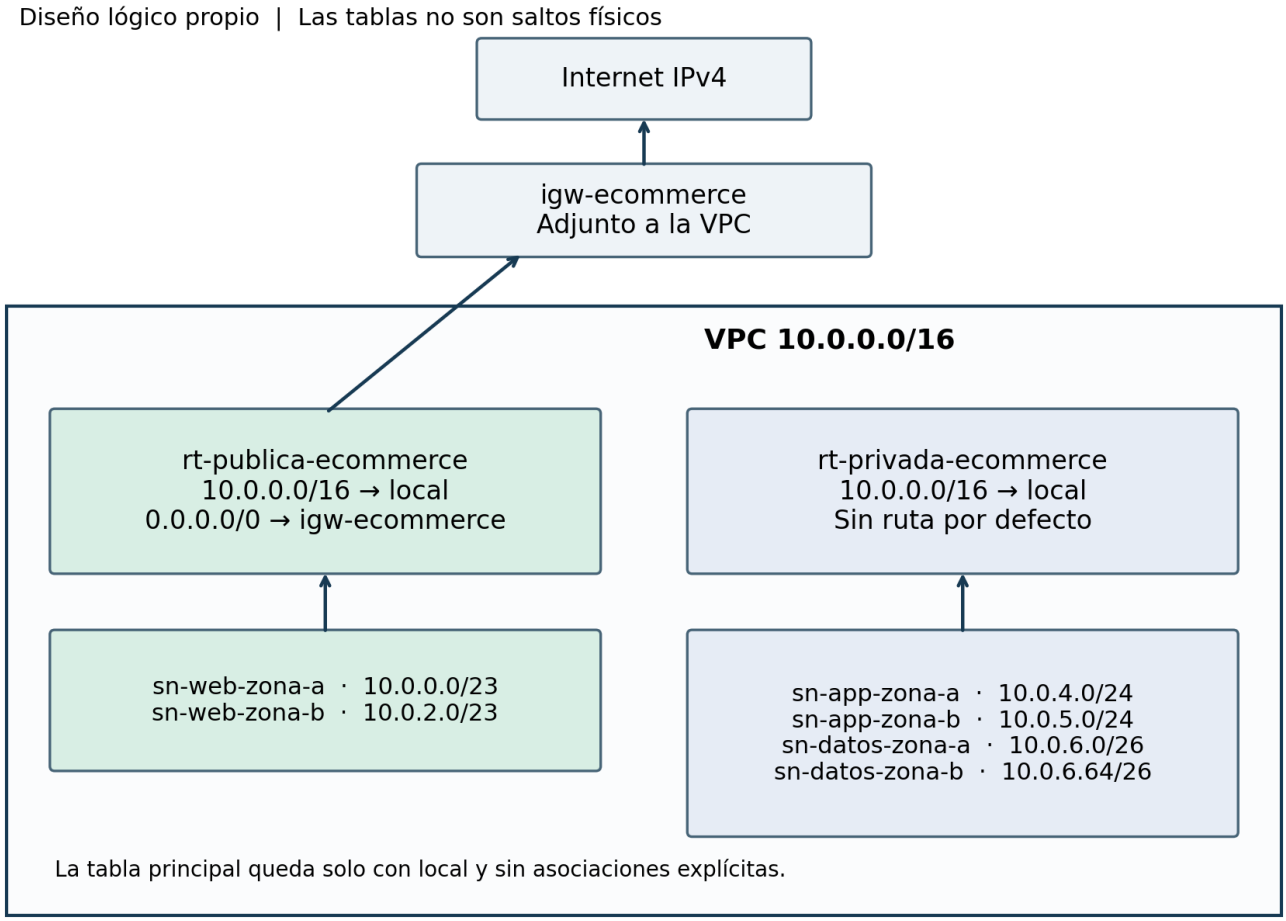


Figura 2. Diagrama lógico propio. Las flechas desde subredes indican asociaciones; la flecha al IGW representa el target de /0. Las tablas son reglas, no dispositivos físicos.

4.3 Decisión para tres destinos de ejemplo

Origen y destino	Coincidencia que gana	Interpretación
Web A → 10.0.4.10	10.0.0.0/16 → local	Tráfico interno; no usa IGW.
Web A → 8.8.8.8	0.0.0.0/0 → IGW	Existe ruta externa; faltan comprobar IP pública y filtros del recurso.
Aplicación A → 8.8.8.8	Ninguna ruta coincide	No existe salida externa en el diseño actual.
Datos B → 10.0.5.10	10.0.0.0/16 → local	Existe ruteo interno entre AZ, sujeto a controles.

Las IP 10.0.4.10 y 10.0.5.10 son ejemplos de destinos dentro de rangos asignables; no identifican instancias reales. El resultado se deduce de las rutas propuestas y de la regla del prefijo más específico. [4]

V. Manual de Route Tables e Internet Gateway

5.1 Procedimiento documentado en consola

Precondición: haber validado el inventario de VPC y subredes. Se trabajaría en us-east-1 y se usaría el mismo VPC ID en todas las operaciones. Las acciones siguientes materializan el diseño de la sección 4; se describen sin ejecutarlas.

1. Crear el Internet Gateway. Abrir VPC → Internet gateways → Create internet gateway. Introducir Name tag = igw-ecommerce y confirmar. Resultado esperado: un ID de IGW y estado Detached antes de adjuntarlo. La existencia del objeto por sí sola no habilita Internet. [6]

2. Adjuntar el gateway. Seleccionar igw-ecommerce → Actions → Attach to VPC. Elegir el VPC ID de vpc-ecommerce-lab y confirmar. Verificación: estado Attached en consola y VPC ID correspondiente. Si el IGW ya está ligado a otra VPC, no intentar reutilizarlo sin revisar la situación. [6]

3. Crear la tabla pública. Abrir Route tables → Create route table. Nombre rt-publica-ecommerce; VPC del proyecto. Crear y abrir Routes. Resultado esperado: 10.0.0.0/16 con target local. No añadir manualmente una segunda copia de la ruta local. [24]

4. Crear la tabla privada. Repetir Create route table con rt-privada-ecommerce en la misma VPC. Mantener solamente 10.0.0.0/16 → local. Verificación: no contiene /0 ni rutas a IGW, NAT, peering o VPN. [24]

5. Añadir la salida pública. En rt-publica-ecommerce → Routes → Edit routes → Add route, indicar Destination = 0.0.0.0/0, Target type = Internet Gateway y seleccionar el ID de igw-ecommerce. Guardar. Resultado esperado: dos rutas, local y por defecto, utilizables y sin blackhole. [6, 24]

6. Asociar las dos subredes web. Abrir Subnet associations → Edit subnet associations de la tabla pública. Marcar exclusivamente sn-web-zona-a y sn-web-zona-b, contrastando sus CIDR. Guardar asociaciones. Resultado esperado: dos asociaciones explícitas. [24]

7. Asociar las cuatro subredes internas. En rt-privada-ecommerce, repetir el proceso para las dos de aplicación y las dos de datos. Guardar. Resultado esperado: cuatro asociaciones explícitas; ninguna web debe aparecer en esta selección. [24]

8. Verificar desde cada subred. Ir a Subnets, filtrar la VPC y abrir Route table en cada fila. Comparar con el mapeo de la siguiente sección. Esta comprobación encuentra errores que podrían pasar inadvertidos si solo se mira la lista de tablas. [8]

9. Cerrar la revisión. Comprobar que la tabla principal continúa con local y que no hay subredes del proyecto dependiendo accidentalmente de ella. Guardar como registro documental el inventario propuesto, las rutas y las asociaciones; no afirmar conectividad de EC2 hasta realizar pruebas en una ejecución futura.

AWS CLI para gateway y tablas

5.2 Crear y adjuntar el Internet Gateway

```
IGW_ID=$(aws ec2 create-internet-gateway \
  --tag-specifications \
  'ResourceType=internet-gateway,Tags=[{Key=Name,Value=igw-ecommerce}]' \
  --query 'InternetGateway.InternetGatewayId' --output text)
aws ec2 attach-internet-gateway \
  --internet-gateway-id "$IGW_ID" --vpc-id "$VPC_ID"
```

create-internet-gateway crea el objeto y devuelve su identificador; attach-internet-gateway vincula ese objeto con la VPC. --internet-gateway-id identifica el gateway, mientras que --vpc-id identifica la red receptora. El segundo comando normalmente no imprime contenido al completarse; eso no sustituye comprobar la asociación con describe-internet-gateways. [25, 28]

5.3 Crear las dos tablas personalizadas

```
RT_PUBLICA=$(aws ec2 create-route-table \
  --vpc-id "$VPC_ID" \
  --tag-specifications \
  'ResourceType=route-table,Tags=[{Key=Name,Value=rt-publica-ecommerce}]' \
  --query 'RouteTable.RouteTableId' --output text)

RT_PRIVADA=$(aws ec2 create-route-table \
  --vpc-id "$VPC_ID" \
  --tag-specifications \
  'ResourceType=route-table,Tags=[{Key=Name,Value=rt-privada-ecommerce}]' \
  --query 'RouteTable.RouteTableId' --output text)
```

Cada create-route-table pertenece a la VPC indicada y devuelve un RouteTableId distinto. AWS incorpora local automáticamente. --tag-specifications exige ResourceType=route-table; usar el tipo vpc aquí sería incorrecto. El nombre es una etiqueta, mientras que RT_PUBLICA y RT_PRIVADA conservan los IDs que usarán los siguientes comandos. [29]

5.4 Agregar la ruta por defecto solo a la tabla pública

```
aws ec2 create-route \
  --route-table-id "$RT_PUBLICA" \
  --destination-cidr-block 0.0.0.0/0 \
  --gateway-id "$IGW_ID"
```

El resultado esperado es Return = true. --route-table-id selecciona dónde escribir la ruta; --destination-cidr-block define el conjunto de destinos y --gateway-id especifica el IGW. No se ejecuta create-route sobre RT_PRIVADA en este laboratorio. Si ya existe /0, se debe inspeccionar antes de decidir una corrección; repetir create-route no reemplaza automáticamente el target existente. [30]

AWS CLI para las seis asociaciones

5.5 Asociar cada subred con su política

```
aws ec2 associate-route-table \
  --route-table-id "$RT_PUBLICA" --subnet-id "$WEB_A"

aws ec2 associate-route-table \
  --route-table-id "$RT_PUBLICA" --subnet-id "$WEB_B"

aws ec2 associate-route-table \
  --route-table-id "$RT_PRIVADA" --subnet-id "$APP_A"

aws ec2 associate-route-table \
  --route-table-id "$RT_PRIVADA" --subnet-id "$APP_B"

aws ec2 associate-route-table \
  --route-table-id "$RT_PRIVADA" --subnet-id "$DATOS_A"

aws ec2 associate-route-table \
  --route-table-id "$RT_PRIVADA" --subnet-id "$DATOS_B"
```

associate-route-table aplica a la subred las rutas de la tabla elegida. --subnet-id corresponde a un ID devuelto por create-subnet, no a su nombre ni a su CIDR. El resultado esperado incluye un AssociationId y un estado de asociación que debe terminar en associated. Ambas entidades deben pertenecer a la misma VPC. [31]

Estos comandos presuponen subredes nuevas con asociación implícita a la principal. Si se corrige una subred que ya tiene asociación explícita, se revisaría su AssociationId y se utilizaría replace-route-table-association, en vez de asumir que repetir associate-route-table es siempre la operación correcta. [8]

Subred	Tabla efectiva al terminar	Ruta externa
sn-web-zona-a	rt-publica-ecommerce	0.0.0.0/0 → IGW
sn-web-zona-b	rt-publica-ecommerce	0.0.0.0/0 → IGW
sn-app-zona-a	rt-privada-ecommerce	Ninguna
sn-app-zona-b	rt-privada-ecommerce	Ninguna
sn-datos-zona-a	rt-privada-ecommerce	Ninguna
sn-datos-zona-b	rt-privada-ecommerce	Ninguna

Verificación: recorrer las seis subredes y comparar su tabla efectiva, no solo el total de asociaciones. Dos asociaciones públicas y cuatro privadas son necesarias, pero también debe coincidir la identidad de cada una.

Parámetros de AWS CLI

Esta tabla explica los parámetros utilizados en los comandos de creación y asociación. Los valores se aplican al diseño propio; la sintaxis se apoya en las referencias de AWS CLI. [17, 22, 25, 28–31]

Parámetro o expresión	Significado en este manual
--cidr-block	Bloque IPv4 de VPC o subred. Para la VPC se usa 10.0.0.0/16; para cada subred, su bloque VLSM exacto.
--instance-tenancy default	Mantiene la opción de tenencia predeterminada de la VPC. No solicita hardware dedicado.
--vpc-id	ID real de la VPC a la que pertenecerán subredes y tablas o a la que se adjuntará el IGW.
--availability-zone	Nombre de la única AZ de cada subred: us-east-1a o us-east-1b en este ejercicio.
--tag-specifications	Etiquetas aplicadas al crear. ResourceType identifica el tipo de recurso; Tags incluye Key=Name y su Value.
--query	Expresión JMESPath para seleccionar datos de la respuesta, por ejemplo Vpc.VpcId. No cambia el recurso.
--output text / table / json	text captura IDs; table facilita lectura; json permite revisar estructuras de rutas y asociaciones.
--internet-gateway-id	ID del IGW para attach-internet-gateway o consultas. No es la IP del router dentro de una subred.
--route-table-id	ID de la tabla que recibirá la ruta o la asociación. En consultas se usa el plural --route-table-ids.
--destination-cidr-block	Prefijo que determina qué destinos coinciden con una ruta. /0 se reserva aquí a la tabla pública.
--gateway-id	Target de tipo gateway usado por create-route; en este diseño contiene el ID del IGW.
--subnet-id	ID real de la subred que se asociará con una tabla.
--filters	Restringe las consultas por campos como vpc-id o attachment.vpc-id.

Parámetro o expresión	Significado en este manual
AWS_DEFAULT_REGION y AWS_REGION	Variables de entorno que fijan us-east-1. Se igualan para evitar una región heredada distinta.
VARIABLE=$(comando)	Sintaxis de Bash que guarda la salida en una variable. Las comillas en "$VARIABLE" conservan el valor como un argumento.

No se incluye --dry-run como supuesto reemplazo de la investigación: esa opción comprueba permisos para las operaciones que la admiten, pero no demuestra que todos los recursos existan ni que el diseño completo tenga conectividad. Tampoco se han utilizado credenciales para ejecutar estas comprobaciones.

VI. Verificaciones técnicas del diseño

6.1 Consultar gateway rutas y asociaciones

```
aws ec2 describe-internet-gateways \
  --internet-gateway-ids "$IGW_ID" \
  --query 'InternetGateways[].{ID:InternetGatewayId,Vinculos:Attachments}' \
  --output json

aws ec2 describe-route-tables \
  --route-table-ids "$RT_PUBLICA" "$RT_PRIVADA" \
  --query 'RouteTables[].{ID:RouteTableId,VPC:VpcId,Rutas:Routes,Asoc:Associations}' \
  --output json

aws ec2 describe-route-tables \
  --filters "Name=vpc-id,Values=$VPC_ID" \
  Name=association.main,Values=true \
  --output json
```

La primera consulta debe mostrar Attachments con el VpcId del proyecto; el valor de State de esa asociación en la respuesta de la API es available, mientras la consola presenta Attached. La segunda debe mostrar las rutas previstas y seis asociaciones explícitas repartidas 2/4. La tercera identifica la tabla principal para confirmar que sigue sin /0. [32, 33]

6.2 Comprobar la asociación de una subred concreta

```
aws ec2 describe-route-tables \
  --filters "Name=association.subnet-id,Values=$WEB_A" \
  --query 'RouteTables[].{ID:RouteTableId,Rutas:Routes}' \
  --output json
```

Se espera rt-publica-ecommerce, identificada por RT_PUBLICA. Si no se devuelve una tabla, no significa que la subred carezca de ruteo: podría usar implícitamente la principal. En ese caso se revisaría la tercera consulta anterior. Para las otras cinco subredes se reemplazaría WEB_A por la variable correspondiente. [8, 33]

6.3 Condiciones de aceptación del ruteo

Objeto	Condición que confirmaría el diseño
IGW	Adjunto a VPC_ID; no a otra red.
Tabla pública	Local y /0 a IGW_ID; ambas rutas sin blackhole.
Tabla privada	Solo local; ningún camino externo añadido.
Asociaciones	Web A/B públicas; aplicación A/B y datos A/B privadas.

Objeto	Condición que confirmaría el diseño
Tabla principal	Local conservada; no proporciona salida pública accidental.

Plan de conectividad sin ejecución real

La Ruta B no exige desplegar EC2 ni aportar terminales ejecutadas. Este plan complementa la investigación: describe qué se probaría si posteriormente existieran instancias autorizadas. Una ruta válida es condición necesaria, pero el resultado también depende de direcciones, seguridad, servicio de destino y sistema operativo. [6, 26, 27]

Prueba propuesta	Resultado esperado	Interpretación y límites
Web → ping 8.8.8.8	Respuesta si ICMP y el destino lo permiten	Verifica ida y respuesta; un fallo aislado no identifica la causa.
Web → traceroute 8.8.8.8	Saltos visibles o asteriscos según respuestas	Complementa el análisis; no garantiza mostrar el router virtual ni el IGW.
Web → IP privada de aplicación	Comunicación si filtros y host permiten ICMP	Se usa local. No es éxito garantizado solo por compartir VPC.
Aplicación → 8.8.8.8	Sin conectividad por ausencia de ruta externa	Corroborar la tabla privada; pérdida de ping por sí sola no demuestra aislamiento.
Web → servicio TCP externo autorizado	Conexión si se cumplen los requisitos	Sirve para distinguir una restricción de ICMP de una falta general de conectividad.

Procedimiento de una comprobación futura

1. Preparar un origen válido. Elegir una EC2 de prueba de la subred web, comprobar su IP pública y conectarse por un método autorizado. Para SSH se limitaría el origen a la IP del administrador. No se asume que una instancia existe hoy.

2. Separar ruteo y filtrado. Revisar tabla efectiva y rutas antes de ejecutar pruebas. Después comprobar reglas de SG, NACL y firewall del host para el protocolo elegido. Una NACL requiere contemplar tráfico de retorno porque es sin estado. [26, 27]

3. Limitar y registrar las sondas. Usar los comandos de la sección 1, indicando IP origen, subred, destino, fecha y protocolo. Documentar la salida auténtica si se ejecutara; no reemplazarla por una salida esperada dibujada.

4. Probar conectividad interna y aislamiento. Repetir desde un origen privado accesible por un mecanismo autorizado. Mantener la ausencia de /0 durante la prueba. No agregar una ruta pública temporal solo para que una prueba privada tenga éxito.

5. Interpretar la evidencia en conjunto. Una respuesta positiva de aplicación prueba más que una simple coincidencia de configuración; un resultado negativo exige identificar dónde se bloqueó. Contrastar configuración y pruebas antes de aceptar o corregir la red.

Errores y diagnóstico orientado a causas

Síntoma o error	Causa que se revisaría	Acción correctiva documentada
InvalidSubnet.Conflict	CIDR superpuesto con otra subred	Comparar límites de ambos bloques y corregir la entrada equivocada antes de crear.
InvalidSubnet.Range	CIDR inválido para una subred	Revisar prefijo, alineación y pertenencia a la VPC. El intervalo general permitido es /16 a /28.
InvalidVpcID.NotFound	ID incorrecto o contexto regional equivocado	Consultar VPC ID y región; no crear otra VPC para ocultar el error.
RouteAlreadyExists	Ya existe una ruta para ese prefijo	Describir la tabla y decidir si debe mantenerse o reemplazarse; no duplicarla.
IGW Detached o VPC incorrecta	Creación sin adjuntar o selección errónea	Verificar Attachments y adjuntar a la VPC prevista antes de configurar /0.
Web sin Internet	Sin /0, asociación equivocada o falta de IP pública	Revisar subred efectiva, target y direccionamiento; después SG, NACL y host.
Ruta blackhole	Target no disponible o conexión no utilizable	Inspeccionar el recurso de destino y su estado; el texto del CIDR no garantiza un camino activo.
Aplicación con ruta directa a IGW	Tabla pública asociada por error	Corregir asociación y revisar cualquier exposición previa de sus recursos.
ping interno sin respuesta	ICMP bloqueado, host sin responder u otra causa	Verificar IP destino, filtros de ambos lados y sistema operativo; local no autoriza el protocolo.

Síntoma o error	Causa que se revisaría	Acción correctiva documentada
traceroute con asteriscos	Respuestas ICMP ausentes o filtradas	Revisar protocolo y controles; comparar con rutas y una conexión TCP autorizada.

Fuentes para los códigos de error: [18]. Para asociaciones, IGW y requisitos de conectividad: [6, 8]. Para ICMP, NACL y SG: [11, 26, 27]. Los síntomas de red no se presentan como equivalentes exclusivos de una sola causa. La corrección concreta se decide después de inspeccionar la configuración real.

VII. Reflexión final y continuidad

7.1 Resultados esperados y aprendizaje

Si ejecutara este procedimiento, esperaría encontrar una VPC 10.0.0.0/16 en estado disponible, con exactamente seis subredes distribuidas entre las dos zonas seleccionadas. No me conformaría con ver seis nombres: compararía los CIDR, las AZ y el identificador de VPC, porque una etiqueta correcta puede acompañar una configuración equivocada. Revisaría también el número de direcciones asignables, especialmente en aplicación, donde el /24 apenas deja una dirección de margen sobre los 250 hosts requeridos.

En la parte de ruteo esperaría dos tablas personalizadas además de la principal automática. La pública debería conservar local y añadir 0.0.0.0/0 hacia el IGW de esta VPC; la privada debería conservar solamente local. Confirmaría las dos asociaciones web y las cuatro internas desde las subredes, no únicamente desde las tablas. Un IGW creado pero no adjunto o una asociación equivocada bastaría para que la intención del diseño no coincida con el comportamiento real. AWS también documenta conflictos de CIDR y rutas duplicadas, por lo que interpretaría esos errores como señales para revisar entradas y dependencias, no como motivos para repetir comandos sin diagnóstico. [6, 8, 18]

Mi conclusión es que la seguridad empieza en el diseño del camino disponible, pero no termina allí. Mantener las capas internas sin salida externa reduce exposición; la ruta local todavía permite plantear comunicación entre las seis subredes y exige reglas de seguridad coherentes. Asimismo, una prueba ping fallida no demuestra por sí sola aislamiento, y una ruta activa no demuestra que un servicio responda. En una ejecución futura combinaría inventario, asociaciones, filtros y pruebas reales. Por ahora el resultado es un diseño reproducible y documentado, no una infraestructura funcionando. Esta diferencia permite presentar una investigación honesta y conservar una base consistente para los laboratorios siguientes.

7.2 Base para los laboratorios 3 y 4

Se conservarán el bloque 10.0.0.0/16, las seis subredes exactas, sus nombres y las dos AZ del registro de arquitectura. Lab 3 ampliará la documentación de seguridad y salida privada sobre esta base; Lab 4 abordará conectividad híbrida sobre el mismo diseño. No se desarrolla ni se despliega aquí ninguna de esas ampliaciones. La VPC y las subredes no se recalcularán ni se recrearán como si fueran una arquitectura distinta.

La continuidad de Ruta B consiste en mantener este inventario y actualizar los diagramas cuando corresponda. No existe una infraestructura real que deba mantenerse encendida ni recursos que borrar al terminar este informe.

VIII. Evidencia documental y revisión de requisitos

8.1 Registro y procedencia de las capturas oficiales

Se incorporan seis capturas auténticas de páginas públicas de AWS Documentation. E1–E3 respaldan el diseño y la creación documentada de VPC y subredes; E4–E6 respaldan tablas de ruteo, consulta de asociaciones e Internet Gateway. Las imágenes muestran documentación, no una consola privada ni una configuración que se haya ejecutado. Se reproducen en las secciones 8.2 a 8.4, con descripción y URL individual.

Evidencia incorporada	Contenido a mostrar	Referencia
E1 • VPC	Página Create a VPC: opciones y procedimientos oficiales para crear una VPC.	[15]
E2 • Subred	Create a subnet: VPC, nombre, AZ y CIDR.	[16]
E3 • CIDR de subredes	Subnet CIDR blocks: bloques dentro de la VPC y ejemplo de subredes sin solapamiento.	[12]
E4 • Tablas	Create the route table y navegación a las secciones de rutas y asociaciones.	[24]
E5 • Asociación	Manage subnet route tables: consulta de la tabla efectiva y de las asociaciones explícitas.	[8]
E6 • IGW	Add internet access to a subnet: creación, asociación del IGW y ruta hacia Internet.	[6]

E1 https://docs.aws.amazon.com/vpc/latest/userguide/create-vpc.html

E2 https://docs.aws.amazon.com/vpc/latest/userguide/create-subnets.html

E3 https://docs.aws.amazon.com/vpc/latest/userguide/subnet-sizing.html

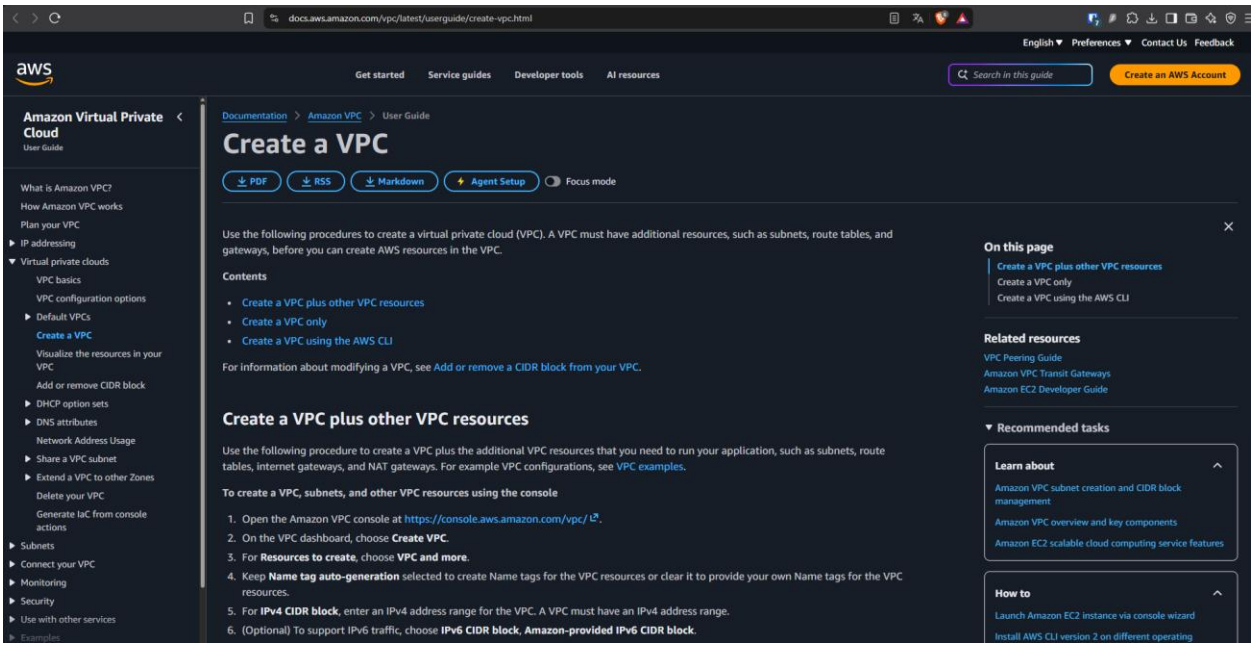
E4 https://docs.aws.amazon.com/vpc/latest/userguide/create-vpc-route-table.html

E5 https://docs.aws.amazon.com/vpc/latest/userguide/WorkWithRouteTables.html

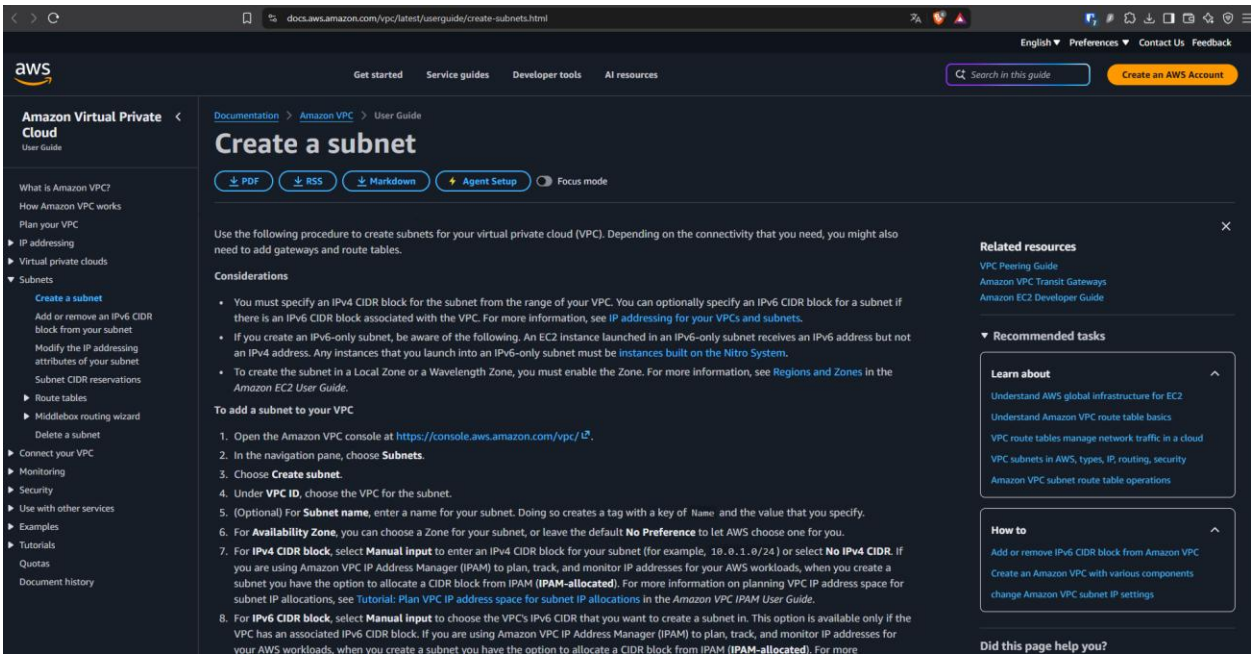
E6 https://docs.aws.amazon.com/vpc/latest/userguide/working-with-igw.html

Registro de las fuentes: imágenes de documentación pública oficial de Amazon Web Services; fecha de incorporación al informe: 11 de octubre de 2026. Sus pies de figura describen únicamente el contenido visible.

8.2 Evidencias oficiales de creación de VPC y subredes

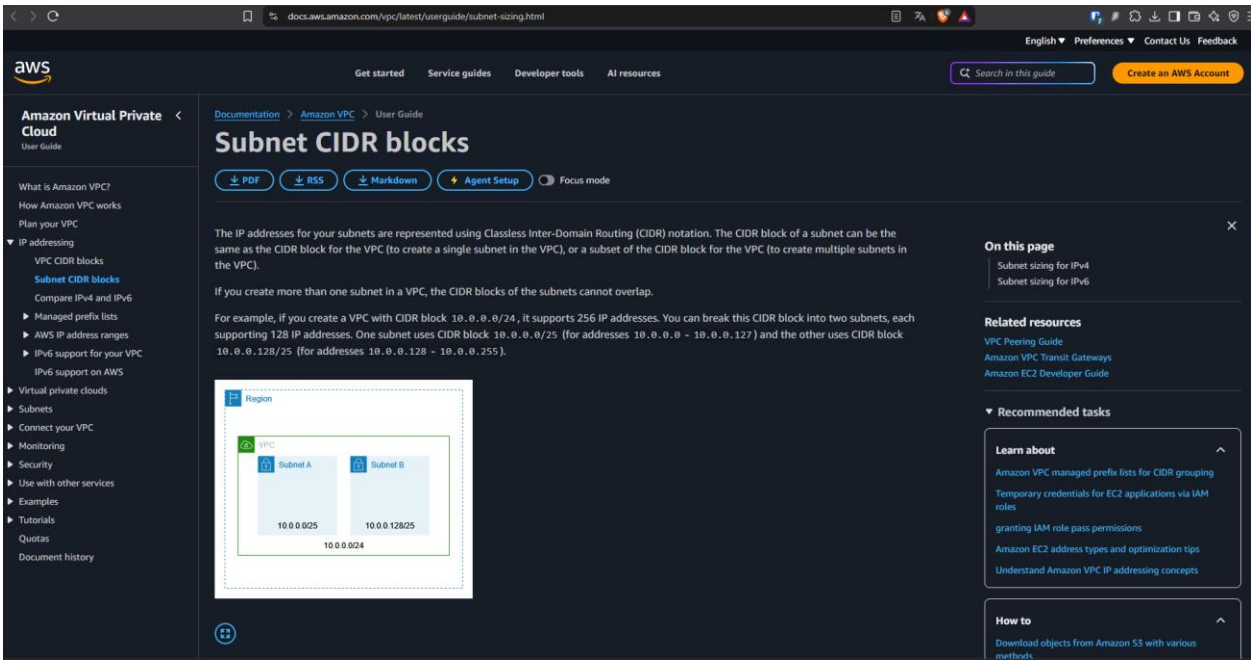


E1 — Create a VPC. Vista de la guía oficial con las opciones para crear una VPC y el acceso al procedimiento «Create a VPC only».
Fuente: AWS Documentation · https://docs.aws.amazon.com/vpc/latest/userguide/create-vpc.html



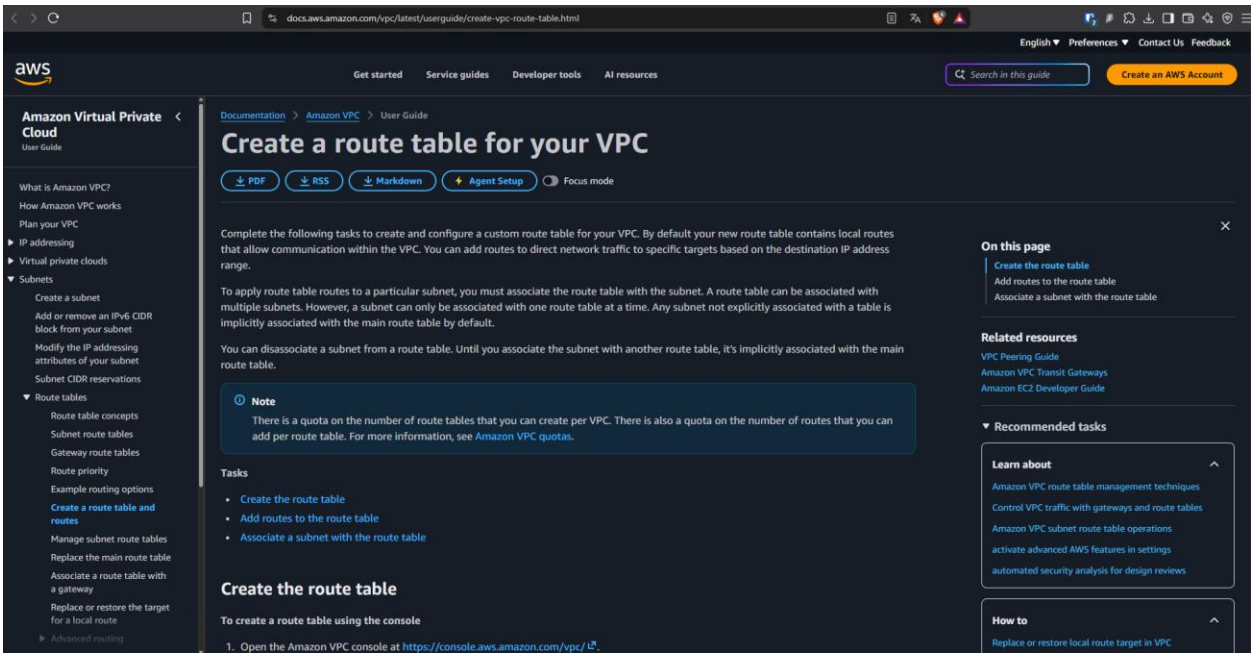
E2 — Create a subnet. Procedimiento oficial para elegir VPC, nombre, zona de disponibilidad y CIDR al crear una subred.
Fuente: AWS Documentation · https://docs.aws.amazon.com/vpc/latest/userguide/create-subnets.html

8.3 Evidencias oficiales de direccionamiento y tablas de ruteo



E3 — Subnet CIDR blocks. Ejemplo gráfico de cómo subdividir un bloque IPv4 de una VPC en subredes sin solapamiento.

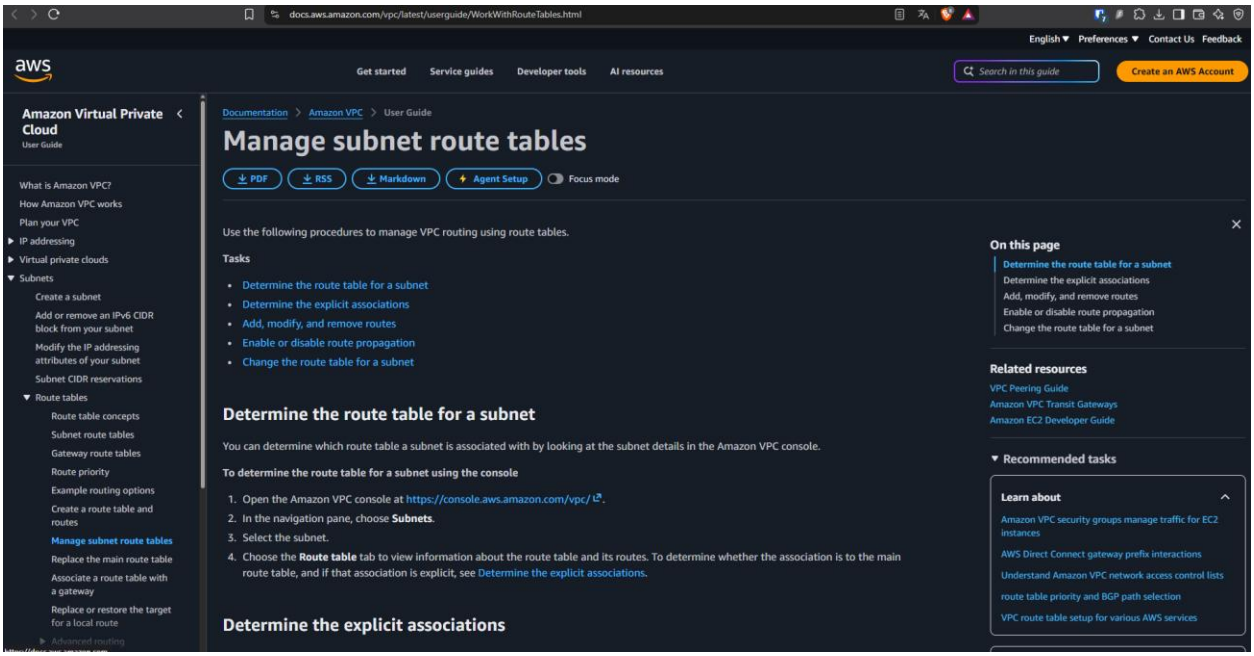
Fuente: AWS Documentation · https://docs.aws.amazon.com/vpc/latest/userguide/subnet-sizing.html ·



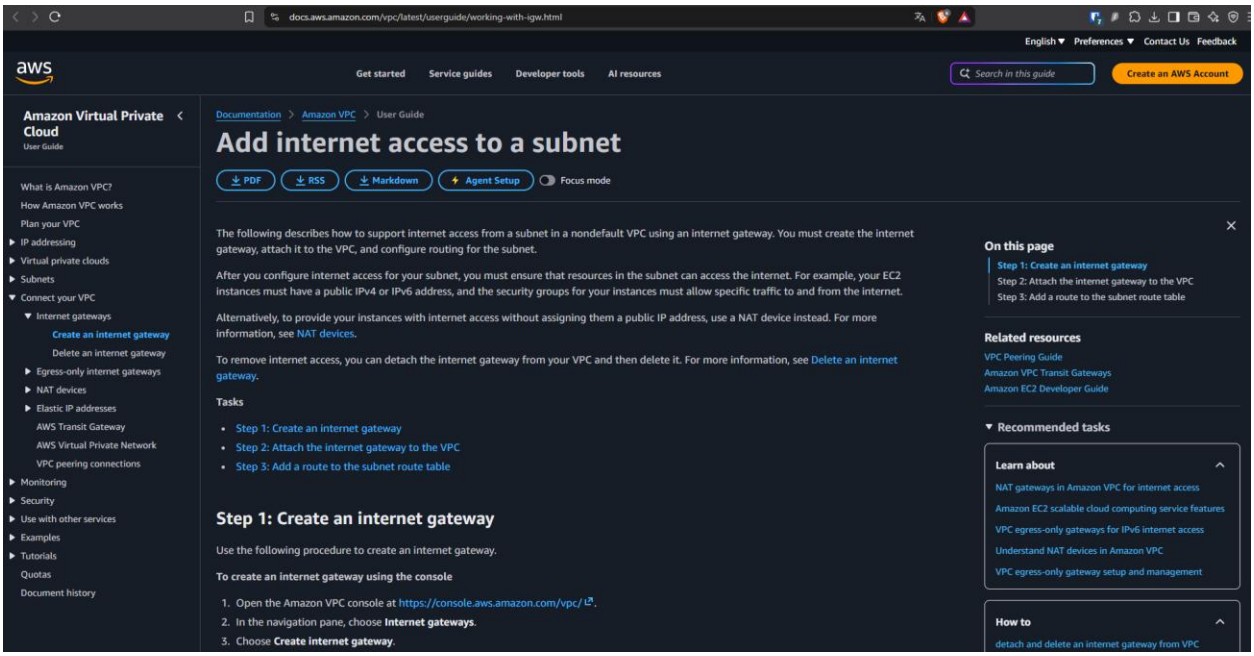
E4 — Create a route table for your VPC. Guía oficial para crear una tabla de ruteo; presenta la ruta local y tareas de agregar rutas y asociar subredes.

Fuente: AWS Documentation · https://docs.aws.amazon.com/vpc/latest/userguide/create-vpc-route-table.html

8.4 Evidencias oficiales de asociaciones y salida a Internet



E5 — Manage subnet route tables. Instrucciones oficiales para determinar la tabla de ruteo efectiva de una subred y consultar sus asociaciones.
Fuente: AWS Documentation · https://docs.aws.amazon.com/vpc/latest/userguide/WorkWithRouteTables.html



E6 — Add internet access to a subnet. Guía oficial que identifica los tres pasos: crear un Internet Gateway, adjuntarlo a la VPC y agregar la ruta.
Fuente: AWS Documentation · https://docs.aws.amazon.com/vpc/latest/userguide/working-with-igw.html

IX. Referencias oficiales

Documentación oficial de Amazon Web Services consultada el 11 de octubre de 2026. Las páginas latest son documentación viva y pueden cambiar después de esta fecha. Las referencias respaldan conceptos y sintaxis; los cálculos y las decisiones aplicadas al caso son desarrollo propio. La guía universitaria adjunta define requisitos, nombres y rúbrica.

[1] Amazon Web Services. Configure route tables.

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html

[2] Amazon Web Services. Subnet route tables.

https://docs.aws.amazon.com/vpc/latest/userguide/subnet-route-tables.html

[3] Amazon Web Services. Replace or restore the target for a local route.

https://docs.aws.amazon.com/vpc/latest/userguide/replace-local-route-target.html

[4] Amazon Web Services. How route priority works.

https://docs.aws.amazon.com/vpc/latest/userguide/route-tables-priority.html

[5] Amazon Web Services. Enable internet access for a VPC using an internet gateway.

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Internet_Gateway.html

[6] Amazon Web Services. Add internet access to a subnet.

https://docs.aws.amazon.com/vpc/latest/userguide/working-with-igw.html

[7] Amazon Web Services. Subnets for your VPC.

https://docs.aws.amazon.com/vpc/latest/userguide/configure-subnets.html

[8] Amazon Web Services. Manage subnet route tables.

https://docs.aws.amazon.com/vpc/latest/userguide/WorkWithRouteTables.html

[9] Amazon Web Services. Update your route tables for a VPC peering connection.

https://docs.aws.amazon.com/vpc/latest/peering/vpc-peering-routing.html

[10] Amazon Web Services. How VPC peering connections work.

https://docs.aws.amazon.com/vpc/latest/peering/vpc-peering-basics.html

[11] Amazon Web Services. Path MTU Discovery and network ACLs.

https://docs.aws.amazon.com/vpc/latest/userguide/path_mtu_discovery.html

[12] Amazon Web Services. Subnet CIDR blocks.

https://docs.aws.amazon.com/vpc/latest/userguide/subnet-sizing.html

[13] Amazon Web Services. How Reachability Analyzer works.

https://docs.aws.amazon.com/vpc/latest/reachability/how-reachability-analyzer-works.html

[14] Amazon Web Services. VPC CIDR blocks.
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-cidr-blocks.html
[15] Amazon Web Services. Create a VPC.
https://docs.aws.amazon.com/vpc/latest/userguide/create-vpc.html
[16] Amazon Web Services. Create a subnet.
https://docs.aws.amazon.com/vpc/latest/userguide/create-subnets.html
[17] Amazon Web Services. AWS CLI create-vpc.
https://docs.aws.amazon.com/cli/latest/reference/ec2/create-vpc.html
[18] Amazon Web Services. Error codes for the Amazon EC2 API.
https://docs.aws.amazon.com/ec2/latest/devguide/errors-overview.html
[19] Amazon Web Services. AWS CLI describe-availability-zones.
https://docs.aws.amazon.com/cli/latest/reference/ec2/describe-availability-zones.html
[20] Amazon Web Services. AWS CLI describe-subnets.
https://docs.aws.amazon.com/cli/latest/reference/ec2/describe-subnets.html
[21] Amazon Web Services. AWS CLI wait vpc-available.
https://docs.aws.amazon.com/cli/latest/reference/ec2/wait/vpc-available.html
[22] Amazon Web Services. AWS CLI create-subnet.
https://docs.aws.amazon.com/cli/latest/reference/ec2/create-subnet.html
[23] Amazon Web Services. Example routing options.
https://docs.aws.amazon.com/vpc/latest/userguide/route-table-options.html
[24] Amazon Web Services. Create a route table for your VPC.
https://docs.aws.amazon.com/vpc/latest/userguide/create-vpc-route-table.html
[25] Amazon Web Services. AWS CLI create-internet-gateway.
https://docs.aws.amazon.com/cli/latest/reference/ec2/create-internet-gateway.html
[26] Amazon Web Services. Control subnet traffic with network access control lists.
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html
[27] Amazon Web Services. Control traffic to your AWS resources using security groups.
https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html
[28] Amazon Web Services. AWS CLI attach-internet-gateway.
https://docs.aws.amazon.com/cli/latest/reference/ec2/attach-internet-gateway.html

[29] Amazon Web Services. AWS CLI create-route-table.
https://docs.aws.amazon.com/cli/latest/reference/ec2/create-route-table.html
[30] Amazon Web Services. AWS CLI create-route.
https://docs.aws.amazon.com/cli/latest/reference/ec2/create-route.html
[31] Amazon Web Services. AWS CLI associate-route-table.
https://docs.aws.amazon.com/cli/latest/reference/ec2/associate-route-table.html
[32] Amazon Web Services. AWS CLI describe-internet-gateways.
https://docs.aws.amazon.com/cli/latest/reference/ec2/describe-internet-gateways.html
[33] Amazon Web Services. AWS CLI describe-route-tables.
https://docs.aws.amazon.com/cli/latest/reference/ec2/describe-route-tables.html